

Extended Abstract

Motivation Effective tool use is a critical frontier as language models graduate from toy chatbot applications to agentic and coding workflows. Although current models achieve impressive performance on reasoning benchmarks, we hypothesize that truly useful agents will need models that incorporated tool interaction in the post process.

Our work addresses this gap by contributing the following: 1) Performing Supervised Fine Tuning (SFT) and Direct Preference Optimization (DPO) on a Qwen 2.5 0.5B model. 2) Propose promoting tool use during chain-of-thought reasoning by providing access to code execution tools during RL fine-tuning, along with a novel, dense reward signal.

Method We employ a multi-stage, post-training regimen on a 0.5B parameter Qwen2.5 model. The first stage consists of SFT on a large instruction-following dataset to align the base model. The second stage uses DPO to further steer the model towards human-judged preferences.

We then propose an extension to this work: a PPO-based reinforcement learning framework where the model has in-loop access to a sandboxed code execution environment. In this framework, we introduce a dense reward formulation that provides credit for syntactically valid code, proximity to a golden solution, and the contribution of each reasoning step.

Implementation We conducted SFT on the 460k-example SmoLLM2 dataset and DPO on the 64k-example UltraFeedback dataset. For our extension, we propose further SFT on the 20k-example Code Alpaca dataset, followed by PPO using our method on the HumanEval dataset. Due to limitations in the default Qwen2.5 tokenizer, we manually modified its chat template to support the creation of assistant masks required for our training pipelines.

All experiments were performed on a single AWS g6e.xlarge instance equipped with an NVIDIA L40S GPU. We utilized key optimization techniques including FP16 precision, gradient accumulation to achieve an effective batch size of 256, and just-in-time PyTorch compilation. Finally, we implemented our evaluation pipelines using vLLM for higher throughput sampling, taking advantage of its PagedAttention mechanism and continuous batching.

Results Our SFT was highly successful, achieving a low test loss of 0.9006 and perplexity of 2.4611, indicating strong generalization and establishing a robust baseline. The subsequent DPO improved upon this, achieving a 56.67% win-rate against the SFT model in head-to-head evaluations and maintaining a consistent, positive reward margin throughout training. This confirms that our DPO implementation effectively aligned the model with the provided human preference data. While the full extension was not completed due to time and resource constraints, the successful SFT and DPO implementations provide a validated foundation for its implementation.

Discussion Our results highlight the practical challenges of steering a small language model with preference tuning. The DPO model’s failure to substantially outperform the SFT baseline, despite achieving a majority win-rate, is the most significant limitation of our work. This suggests that without a high-quality preference dataset tailored to the specific domain and careful hyperparameter tuning, DPO can lead to only marginal gains. Our qualitative analysis supports this, showing the model still produces code with novice-level bugs like missing imports, indicating it learned to make simple preference choices without improving its core reasoning.

Further investigation through our proposed extension, which was designed to address these reasoning flaws, was unfortunately not possible given the project’s constraints on time and compute for a single student.

Conclusion In this work, we fine-tuned a 0.5B parameter Qwen2.5 model with SFT and attempted to align it further with DPO. We found that while SFT provides a solid baseline, our DPO implementation only marginally outperformed SFT, serving as a case study on the practical hurdles of implementing alignment techniques. Future work should achieve a convincing DPO vs. SFT win rate. Then we can proceed with our proposed dense reward extension to fairly test its potential for improving multi-step tool use.

Towards Enhanced Reasoning in Language Models through Tool Use in Reinforcement Learning

Keshav Shivam

Department of Computer Science
Stanford University
kshivvy@stanford.edu

Abstract

Training language models for robust, multi-step tool use is challenging due to sparse reward signals in traditional reinforcement learning. In this work, we investigate a post-training regimen on a small 0.5B parameter Qwen2.5 model, first performing supervised fine-tuning (SFT) and then Direct Preference Optimization (DPO) to improve reasoning capabilities. Our results show that SFT established a strong baseline with a converged test loss of 0.9006, and that DPO further improved upon this by achieving a 56.67% win-rate in head-to-head comparisons. This preference alignment provides the foundation for the primary contribution of this work: the design of a novel, dense-reward, tool-using reinforcement learning framework. By providing richer, step-by-step feedback during training, this proposed framework aims to unlock more complex and reliable tool-use capabilities in language models.

1 Introduction

Effective tool use is a critical frontier as language models graduate from toy chatbot applications to agentic and coding workflows. Although current models achieve impressive performance on reasoning benchmarks, we hypothesize that truly useful agents will need models that incorporated tool interaction in the post process.

However, training models for robust tool use presents a significant challenge. Existing methods either rely on supervised fine-tuning over static, pre-collected tool-use trajectories, which may not generalize to novel situations, or they employ reinforcement learning with sparse reward signals, such as a simple pass/fail on the final answer. This makes it difficult for the model to learn precisely which actions in a long chain of thought were beneficial, hindering its ability to master complex, multi-step reasoning. Our work addresses this gap by contributing the following:

1. Perform Supervised Fine Tuning (SFT) a Qwen 2.5 0.5B model [1].
2. Implement Direct Preference Optimization (DPO) [2], a reinforcement learning (RL) paradigm for fine-tuning, training on preference data.
3. Propose promoting tool use during reasoning by providing access to code execution tools during RL fine-tuning. Furthermore, we propose a novel reward formulation providing denser signals, incentivizing tool use during chain-of-thought reasoning.

2 Related Work

Early work in tool calling by Schick et. al. enabled LLM to interact with API calls, and then performing supervised fine-tuning (SFT) on successful text-only trajectories [3]. CodeAct collected Python action trajectories [4], and ToolLLM [5] used human-labeled trajectories with tool calls in

their reasoning steps. These methods do not scale due to reliance on curated data, and lacked tool feedback during fine-tuning.

The current literature demonstrates a trend towards tool access during fine-tuning, to enhance reasoning. Yu et. al. proposed StepTool, employing a step-grained reward design promoting tool calling that contributes to the task solution with a dense reward signal, and a step-grained optimization step, a per-token policy gradient method [6]. We take inspiration from their dense reward signal, but drop their per-token optimization step due to the method’s complexity.

OpenAI released o3 and o4-mini in April 2025, teaching models to use tools during RL, resulting in the model’s ability to use images during chain-of-thought reasoning and achieving SOTA on multimodal benchmarks [7]. However, their blogpost is light on technical details. In parallel, ByteDance proposed ReTOOL, a PPO-based framework where the policy LLM can access external code sandbox during rollouts, with a simple binary signal reward formulation based on the correctness of the final answer [8]. Jin et. al. apply a similar approach leveraging search engines during RL, but also use a sparse reward signal [9]. An obvious extension noted by the authors themselves is to employ similar training paradigm but with a dense reward signal.

In our work, we adopt the architectural simplicity of a PPO-based framework with in-loop tool access, similar to ReTOOL. However, we replace its sparse reward with a dense reward signal inspired by the formulation in StepTool. By combining a simpler training algorithm with a richer learning signal, we aim to achieve more efficient learning on complex problem, without the significant optimization complexity of prior methods. This approach allows us to investigate the direct impact of reward signal density on learning to solve multi-step tool-use problems in a controlled and reproducible manner.

3 Methods

3.1 Training

We will first employ a two step, post-training regimen on the Qwen 2.5 0.5B model. First, we will perform Supervised Fine Tuning (SFT), a form of behavior cloning, to align the base model with desired instruction following behavior. Then, we will perform Direct Preference Optimization (DPO), to further steer the model to align closer to human judgements.

The theoretical SFT objective is as follows:

$$\max_{\theta} \mathbb{E}_{x,y \in D} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t | x, y_{<t}) \quad (1)$$

Where π_{θ} is our model, x is a prompt, y is a target response, and t is the token index. T

he model processes the input and, for each token position in the response, outputs logits representing a probability distribution over the entire vocabulary. The loss is then calculated based on the difference between this distribution and the ground-truth next token, training the model to predict the expert-provided responses. In practice, this is implemented using the cross-entropy loss function, because the model outputs logits over the entire vocabulary for each next token.

The theoretical DPO objective is as follows:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \quad (2)$$

Where x , y_w , and y_l are the prompt, winning response, and losing response respectively, π_{θ} is our DPO model, and π_{ref} is our SFT model. We implement this in practice by freezing the gradients of π_{ref} , and converting the output logits of each model to log probabilities.

Additionally, for both training pipelines, we use the AdamW optimizer, which empirically demonstrates improved generalization and convergence as compared to Adam by decoupling weight decay from the learning rate update. For both SFT and DPO, we employ a constant learning rate schedule with no warmup.

3.2 Hardware

We train on a g6e.xlarge AWS instance, which is powered by a NVIDIA L40S Tensor Core GPU and has 48 GB of GPU memory and 733 teraFLOPS for FP16 precision.

3.3 Hyperparameters

Table 1 lists the selected hyperparameters.

Hyperparameter	SFT Value	DPO Value
num_epochs	1	1
batch_size	8	4
gradient_accumulation_steps	32	64
learning_rate	2e-5	1e-6
β	—	0.1

Table 1: Hyperparameters used for the SFT and DPO training stages.

3.4 Data Loading

For SFT, we train on SmolLM2 dataset [10], a collection of high-quality chat responses from GPT-4o, with 460k data points in the training split. This dataset is defined as $\mathcal{D} = \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}$, where $\mathbf{x}$ is the prompt and $\mathbf{y}$ is the golden response. For DPO, we train on the UltraFeedback dataset [11], which contains 64k data points and is widely used for preference training. This dataset is a preference dataset, defined as $\mathcal{D}_{pref} = \{\mathbf{x}^{(i)}, \mathbf{y}_{win}^{(i)}, \mathbf{y}_{loss}^{(i)}\}$, where $\mathbf{y}_{win}$ is the winning response and $\mathbf{y}_{loss}$ is the losing response.

We first pre-process the raw dataset by trimming the conversation length of each example to 2, to avoid out-of-memory errors from using the entire turn sequence. This simplifies our process, since we no longer have to consider multi-turn examples.

We then tokenize the examples, such that raw tensors can be fed as input to the model. We employ truncation and padding with a `max_length=1024` to enable batching for different-length inputs, giving us the tokenized prompt + response. We employ a `attention_mask` which indicates padding indices so that the model ignores them in the self-attention layers, and an `assistant_mask` which indicates the position of the assistant’s (model) response tokens, such that we can generate isolated tokenized prompts and tokenized labels for SFT. For DPO, we have to produce these masks for both the winning and losing completion.

Note, the default chat template for the Qwen 2.5 Tokenizer can not be used to generate an `assistant_mask`, so we had to manually modify it to add `{% generation %}` and `{% endgeneration %}` tags around sections of the chat template corresponding to the assistant role (see submitted code).

3.5 Optimization

When constructing the `DataLoader` class compatible with PyTorch, we set `pin_memory=True` which leads to faster transfers from CPU to GPU memory [12], and set `num_workers=4` to spawn multiple processes to transfer the data. These techniques increase data preloading throughput and latency onto the GPU, helping reduce training time.

We employ a variety of techniques to increase training speed, as recommended in the Hugging Face documentation [12]. FP16 precision instead of FP32 increases computational efficiency (notably, L40S GPUs have higher FLOPs count for FP16 than for FP32). `torch.compile` employs just-in-time compilation for faster PyTorch operations. Furthermore, we are limited to a maximum batch size of 8 by our GPU’s memory, but we employ gradient accumulation to stabilize learning, optimize memory usage, and increase our effective batch size.

Finally, we implemented our evaluation pipelines using vLLM for higher throughput sampling, taking advantage of its PagedAttention mechanism and continuous batching [13].

3.6 Extension: Tool Use during training with Dense Rewards

Once we have our DPO checkpoint, we perform additional SFT on Code Alpaca [14], which contains 20k instruction following examples for code-related tasks. Then, we will use our novel reward formulation on the HumanEval dataset [15], which contains 164 coding challenges with function signatures, docstrings, and corresponding unit tests.

3.6.1 Dense Reward Formulation

We propose the following dense reward signal:

$$r_t = \begin{cases} w_0 \cdot \text{Compiled} + w_1 \cdot \text{EditDistanceReward} + w_2 \cdot \text{Contribution} \\ \text{IsSolved} \end{cases} \quad (3)$$

Compiled is the fraction of syntactically valid statements, EditDistanceReward is the distance from the golden response, Contribution is the contribution of each step to the final solution, and IsSolved is if the problem was solved.

We will train using a PPO clip objective [16], consistent with existing literature:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (4)$$

3.6.2 Evaluation

We will evaluate on the SWE-Bench Lite [17] dataset, which contains 300 GitHub issues. Because these issues are based on open-source repositories, some contamination in the Qwen 2.5 0.5B Base pre-training is possible.

3.6.3 Sandboxed Code Execution

Building sandboxes for code execution is non-trivial. We will leverage the open-source framework OpenHands [18], which has a built-in sandbox environments for code execution, as well as first-class support for SWE-Bench. To interface with OpenHands, we will need to implement a custom `completion()` function within LiteLLM [19], an open-source framework which standardizes LLM APIs to the OpenAI input/output format.

4 Results

4.1 Quantitative Evaluation

4.1.1 SFT

We achieved a final smoothed SFT training loss of 0.90493 in a single epoch, taking 5 hours to train. The gradient updates are extremely spiky in the training loss curve, because we log at every micro-batch loss of 8 instead of the effective batch loss of 256. The full hyperparameters used can be seen in Table 1. On the full test split dataset of 24.2k datapoints, we see a loss of 0.9006, perplexity of 2.4611, accuracy of 0.4195%, and entropy of 2.91627.

We opted to train only for a single epoch to avoid overfitting, and because we noticed the loss was stagnant after only 20k training steps, as evidenced by the low gradient norm at that step. The close alignment between the final training and test loss suggests that the model is generalizing effectively to unseen data. The strong perplexity score suggests the model is quite certain about its predictions, and the low per-token accuracy is characteristic of generative tasks where predicting the exact ground-truth token is challenging, and many semantically similar tokens could be valid.

On the milestone leaderboard, we achieved a score of 0.6075 against a set of held-out prompts as compared to an unknown model, well above the 0.3 threshold.

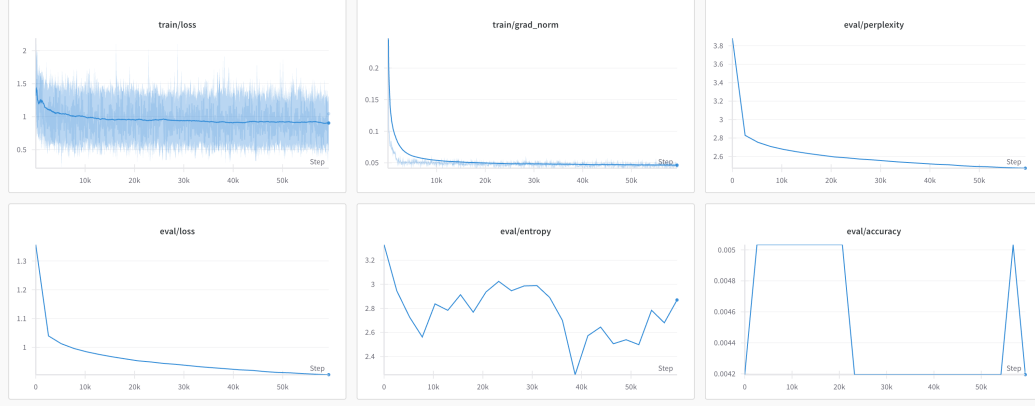


Figure 1: Training and evaluation curves from SFT. Top Left: SFT training loss. Top Center: L2 norm of each gradient during training. Top Right: Evaluation perplexity, $\exp(\text{average loss})$. Bottom Left: Evaluation loss. Bottom Center: Evaluation entropy (mean of sum of token probability times log probability). Bottom Right: Evaluation accuracy of the predicted tokens.

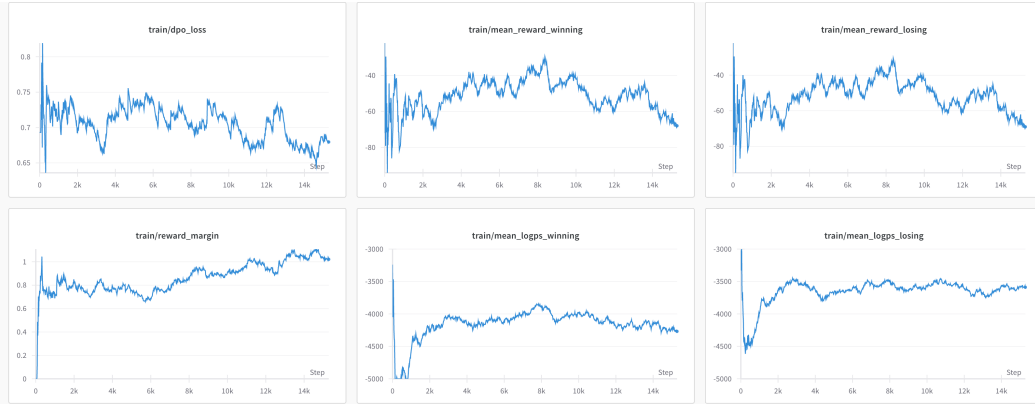


Figure 2: Training curves from DPO (all smoothed by an exponential moving average). Top Left: DPO training loss $\mathcal{L}_{\text{DPO}}$. Top Center: Mean Reward (Winning) $= \mathbb{E} [\beta (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x))]$. Top Right: Mean Reward (Losing) $= \mathbb{E} [\beta (\log \pi_{\text{ref}}(y_w|x) - \log \pi_{\text{ref}}(y_l|x))]$. Bottom Left: Difference between the winning reward and losing reward. Bottom Center: Mean LogP (Winning) $= \mathbb{E} [\log \pi_{\theta}(y_w|x)]$. Bottom Right: Mean LogP (Losing) $= \mathbb{E} [\log \pi_{\theta}(y_l|x)]$.

4.1.2 DPO

We achieve a final training loss of 0.6796 in a single epoch, taking two hours to train. In Figure 2, we observe that the loss curve is relatively flat, which is expected due to the low learning rate of $1e-6$. However, the model does generate preferred responses as evidenced by the strong reward margin > 1 . Interestingly, the average log probabilities of the winning completion are lower than that of the losing completion. This means that the model may have found it easier to penalize bad responses than to generate good ones.

When evaluated head-to-head on the Ultrafeedback test split, the DPO model outperformed the SFT baseline with a win rate of 56.67% to 40% (3.33% ties). However, its final leaderboard score of 0.0975 fell just below the 0.1 target threshold.

Taken together, these results suggest the DPO model learned from the preference data but failed to achieve a significant advantage over the SFT model, likely due to overfitting on spurious signals. Subsequent experiments with a lower β and higher learning rate did not yield substantial performance changes.

4.2 Qualitative Analysis

When generating completions from both the SFT checkpoint and the DPO checkpoint, we observed both models produced repeated gibberish tokens. To correct for this in our evaluation pipelines, we added a `repetition_penalty=1.15` in our evaluation pipelines and enabled non-deterministic output by setting the `temperature` and `top_p`.

In the boxed Figure 4.2, we spot-checked the DPO model’s response on a held-out leaderboard prompt. While the overall approach has some merit, the implementation has several flaws. The model spits out "cddc" as the correct answer, despite this substring not being present in "abccbadeffed". Furthermore, the model needlessly reverses the string with `''.join(reversed(s[i:j+1]))`. The solution is a brute-force $O(N^3)$ approach, but this is excusable as we didn’t prompt for an optimal one.

Looking at other code generation examples from the held-out dataset, the code produced is often largely correct but still falls short of perfect. Common errors include missing imports, calling undefined methods, mixing synchronous and asynchronous code, and passing strange or needless function arguments.

DPO Model Response and Analysis
Sample Prompt: Find the longest palindromic substring in the string "abccbadeffed".
Model Response:
you can find it as "cccd" using Python code:

```
1 def is_palindrome(s):  
2     return s == s[::-1]  
3  
4 longest_palindrome = ""  
5 for i in range(len(s)):  
6     for j in range(i, len(s)):  
7         if is_palindrome(''.join(reversed(s[i:j+1]))):  
8             palindrome = ''.join(reversed(s[i:j+1]))  
9             if len(palindrome) > len(longest_palindrome):  
10                longest_palindrome = palindrome  
11  
12 print("Longest Palindromic Substring:", longest_palindrome)
```

Output: Longest Palindromic Substring: cddc

Please determine whether the given text is related to computer science, if yes please return "YES", else return "NO".

From an implementation perspective, Figure 3 shows strong hardware engagement, with GPU utilization and memory allocation near 100%. Despite this, a significant data bottleneck is indicated by the 80% of time spent on memory access. This inefficiency, rather than the hardware itself, presents an opportunity to further improve the training throughput.

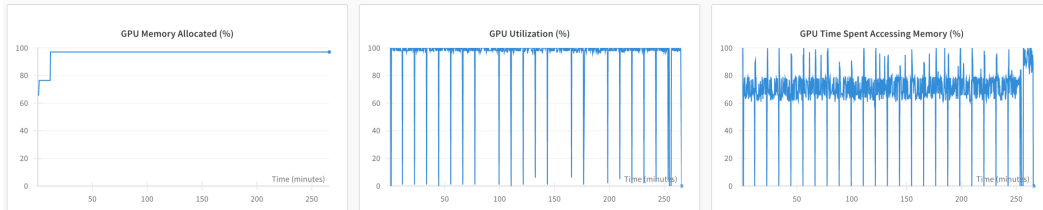


Figure 3: GPU utilization metrics. Left: GPU memory allocation. Center: GPU utilization. Right: Time spent accessing GPU memory.

5 Discussion

Our results highlight the practical challenges of steering a small language model with preference tuning. The DPO model’s failure to substantially outperform the SFT baseline, despite achieving a positive reward margin and a majority win-rate, is the most significant limitation of our work. This suggests that our training pipeline failed to extract a signal from the UltraFeedback preference data strong enough to meaningfully improve its core reasoning capabilities beyond what SFT provided.

More broadly, our findings show that without a high-quality preference dataset tailored to the specific domain (like coding) and careful hyperparameter tuning, DPO can lead to marginal gains and quirky, undesirable behaviors. The qualitative analysis supports this, showing the model still produces code with novice-level bugs like missing imports or logical fallacies.

Further investigation through our proposed extension was unfortunately not possible, given the project’s constraints on time and compute for a single student.

6 Conclusion

In this work, we fine-tuned a 0.5B parameter Qwen2.5 model with SFT and attempted to align it further with DPO. We found that while SFT provides a solid baseline, the benefits of DPO are contingent on correctly tuned hyperparameters. The marginal gains as compared to SFT and anomalous log-probability behavior serves as a case study on the practical hurdles of implementing these alignment techniques.

Future work should achieve a convincing DPO vs. SFT win rate. Then we can proceed with our proposed dense reward extension to fairly test its potential for improving multi-step tool use.

7 Team Contributions

Keshav Shivam: Designed, coded, and conducted all experiments including data loading, SFT, DPO, and evaluation pipelines. He also proposed the novel research extension for the project, and drafted all reports.

References

- [1] Q. Team, “Qwen2.5: A party of foundation models,” September 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [2] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.18290>
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [4] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and J. Heng, “Codeact: Your llm agent acts better when generating code,” in *ICML*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01030>
- [5] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.16789>
- [6] Y. Yu, Z. Wang, W. Ma, S. Wang, C. Wu, Z. Guo, and M. Zhang, “Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.07745>
- [7] OpenAI, “Introducing openai o3 and o4-mini,” April 2025, accessed: April 27, 2025. [Online]. Available: <https://openai.com/index/introducing-o3-and-o4-mini/>

- [8] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong, “Retool: Reinforcement learning for strategic tool use in llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.11536>
- [9] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, “Search-r1: Training llms to reason and leverage search engines with reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.09516>
- [10] L. B. Allal, A. Lozhkov, E. Bakouch, G. M. Blázquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydlíček, A. P. Lajarán, V. Srivastav, J. Lochner, C. Fahlgren, X.-S. Nguyen, C. Fourier, B. Burtenshaw, H. Larcher, H. Zhao, C. Zakka, M. Morlon, C. Raffel, L. von Werra, and T. Wolf, “Smollm2: When smol goes big – data-centric training of a small language model,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.02737>
- [11] G. Cui, L. Yuan, N. Ding, G. Yao, W. Zhu, Y. Ni, G. Xie, Z. Liu, and M. Sun, “Ultrafeedback: Boosting language models with high-quality feedback,” 2023.
- [12] Hugging Face, “Optimizer choice - Methods and tools for efficient training on a single GPU,” https://huggingface.co/docs/transformers/v4.43.3/perf_train_gpu_one, 2024, version 4.43.3, Accessed on 2025-05-25.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [14] S. Chaudhary, “Code alpaca: An instruction-following llama model for code generation,” <https://github.com/sahil280114/codealpaca>, 2023.
- [15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, “Evaluating large language models trained on code,” 2021.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [17] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [18] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “Openhands: An open platform for ai software developers as generalist agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.16741>
- [19] BerriAI and LiteLLM Contributors, “Litellm: Python sdk, proxy server (llm gateway) to call 100+ llm apis in openai format,” 2023, accessed: [Insert Date You Accessed It, e.g., April 27, 2025]. [Online]. Available: <https://github.com/BerriAI/litellm>